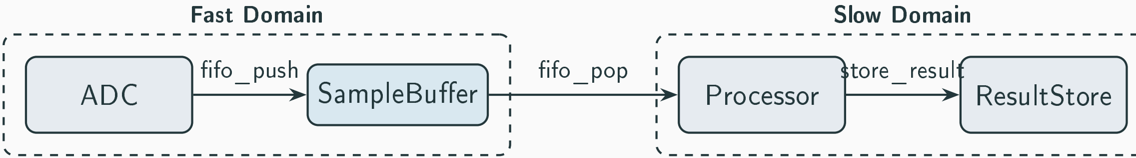
04 — Cross-Domain Composition

Multiple Clock Domains

Mariano Cerrutti

Mununu Tutorial

Architecture: Fast + Slow Domains



- **Fast domain:** ADC samples at high rate, pushes to buffer.
- **Slow domain:** Processor reads from buffer, stores results.
- **FIFO:** Shared labels enforce cross-domain synchronization.

Component Automata

ADC — 3 states, pushes to buffer:

```
        label adc_done;
        label adc_skip;
    }

    states {
        state AdcIdle initial;
        state Converting;
        state AdcDone;
    }

    transitions {
        // Start conversion
        transition AdcIdle -> Converting on label adc_start;
        // Conversion complete
        transition Converting -> AdcDone on label adc_done;
        // Push result to FIFO (synchronizes with SampleBuffer)
```

Shared-Label Synchronization

Three shared labels create synchronization barriers:

Label	Syncs	Purpose
`fifo_push`	ADC ↔ SampleBuffer	Fast domain hand-off
`fifo_pop`	SampleBuffer ↔ Processor	Cross-domain transfer
`store_result`	Processor ↔ ResultStore	Slow domain hand-off

```
controllable {
    label store_skip;
}

states {
    state StoreEmpty initial;
    state StoreHolding;
}
```

Verification Results

Composed state space: $3 \times 2 \times 3 \times 2 = 36$ states.

```
                // Receive from Processor (synchronizes with Processor)
                transition StoreEmpty -> StoreHolding on label store_result;
                // Clear result (external read)
                transition StoreHolding -> StoreEmpty on label store_clear;
                // Skip: store idles this cycle
                transition StoreEmpty -> StoreEmpty on label store_skip;
                transition StoreHolding -> StoreHolding on label store_skip;
            }
        }
    }

    composition {
        // Step 1: Compose each clock domain synchronously
        synchronous fast_domain {
            members [ADC, SampleBuffer];
        }
```

Key Takeaways

1. **Shared labels = synchronization barriers.**
   Two automata sharing a label must take that transition together.
2. **Non-shared labels = independent steps.**
   `adc_start` only involves ADC; others are unaffected.
3. **Flat composition models hierarchy.**
   Without nested composition, shared labels partition the system into logical domains.
4. **State space is the full product.**
   36 states for four small components — cross-domain systems grow quickly.